



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y
DISEÑO DIGITAL
CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

Gestión de Estado: Sesiones y Carrito de Compras
Implementación comparativa en PHP 8, ASP.NET Core 8 y Java/Spring
Boot 3

ALUMNO:

CRUZ PEREZ JUSTYN KEITH

REPOSITORIO DE GITHUB:

github.com/keithdrox/practica-sesiones-CRUZ

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERON

MATERIA:

APLICACIONES WEB - A - [5to Nivel]

Quevedo - Ecuador
2026

1. Descripción de la Práctica

La presente práctica de laboratorio tiene como objetivo principal implementar un carrito de compras mínimo empleando sesiones del lado del servidor. Para comprender de forma integral este mecanismo de gestión de estado, el mismo flujo se ha implementado de forma comparativa en tres plataformas tecnológicas distintas: **PHP 8**, **ASP.NET Core 8** y **Java / Spring Boot 3**.

El protocolo HTTP es inherentemente sin estado, por lo que las sesiones permiten al servidor recordar las solicitudes anteriores de un usuario mediante un identificador único (`session_id`) enviado a través de cookies. Esta práctica evidencia las diferencias en las APIs, configuraciones y mecanismos de seguridad de cookies entre los diferentes lenguajes.

2. Capturas de Pantalla de Implementación

A continuación, se presentan las capturas de pantalla que evidencian el funcionamiento del carrito de compras y la inspección de la seguridad de las cookies en el navegador (DevTools) para cada plataforma.

2.1. Plataforma 1: PHP 8



Figura 1: PHP 8: Funcionamiento del carrito de compras.

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite
PHPSESSID	k8jf3j9fj39fj39...	localhost	/	Session	32	✓	✓	Strict

Figura 2: PHP 8: Verificación de flags `HttpOnly` y `Secure` en la cookie `PHPSESSID`.

2.2. Plataforma 2: ASP.NET Core 8

Tienda - Sesion ASP.NET Core

Catalogo

Laptop - \$450 ▾

Agregar al carrito

Carrito

Producto	Precio	Accion
Laptop	\$450	Eliminar
Mouse	\$25	Eliminar

Total: \$475

Vaciar carrito

Session ID: `aspnetcore.session.123`

Figura 3: ASP.NET Core 8: Funcionamiento del carrito de compras MVC.

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite
.AspNetCore.Session	CfD38B...	localhost	/	Session	32	✓	✓	Strict

Figura 4: ASP.NET Core 8: Verificación de seguridad en la cookie `.AspNetCore.Session`.

2.3. Plataforma 3: Java / Spring Boot 3

Tienda - Sesion Spring Boot

Catalogo

Laptop - \$450 ▾

Agregar al carrito

Carrito

Producto	Precio	Accion
Laptop	\$450	Eliminar
Mouse	\$25	Eliminar

Total: \$475

Vaciar carrito

Session ID: `springboot_sess_123`

Figura 5: Spring Boot 3: Funcionamiento del carrito mediante Thymeleaf.

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite
CARRITO_SESSION	aB3xK9mPqR7s...	localhost	/	Session	32	✓	✓	Strict

Figura 6: Spring Boot 3: Verificación de seguridad en la cookie `CARRITO_SESSION`.

3. Requisitos Implementados y Comparativa

3.1. Tabla Comparativa de Plataformas

Se completó el análisis de la gestión de estado evidenciando las siguientes diferencias clave:

Criterio	PHP 8	ASP.NET Core 8	Spring Boot 3
Iniciar sesión	<code>session_start()</code> al inicio de cada script.	<code>AddSession()</code> y middleware <code>UseSession()</code> .	Automático mediante <code>HttpSession</code> inyectada.
Guardar objeto	Arreglo nativo <code>\$_SESSION[k]</code> .	Requiere serializar a JSON + <code>SetString()</code> .	Directo: <code>session.setAttribute(k, obj)</code> .
Destruir sesión	<code>session_destroy()</code> .	<code>Session.Clear()</code> .	<code>session.invalidate()</code> .
Seguridad Cookie	Configurado con <code>ini_set()</code> (<code>HttpOnly</code> , <code>Secure</code>).	<code>CookieSecurePolicy</code> y opciones de sesión.	Configuración vía <code>application.yml</code> .
Almacenamiento	Archivos en disco (<code>/tmp</code>) por defecto.	Memoria (vía <code>AddDistributedMemoryCache()</code>).	Memoria JVM (o Redis si está configurado).
Serialización	Automática (<code>serialize()</code>).	Manual usando <code>JsonSerializer</code> .	Automática (requiere interfaz <code>Serializable</code>).

Cuadro 1: Comparativa de manejo de sesiones en las tres plataformas.

3.2. Seguridad Implementada

Para garantizar la integridad y seguridad de la sesión de los usuarios, se configuraron estrictamente las cookies de sesión en las tres plataformas aplicando las siguientes directivas:

- **HttpOnly:** Previene ataques de tipo XSS al prohibir que código JavaScript del lado del cliente pueda acceder al identificador de la cookie.
- **Secure:** Asegura que la cookie viaje únicamente a través de canales cifrados HTTPS.
- **SameSite=Strict:** Restringe el envío de la cookie en solicitudes de origen cruzado, mitigando fuertemente el riesgo de ataques CSRF.

3.3. Patrón Post/Redirect/Get (PRG)

En todas las plataformas, los flujos que alteran el estado del servidor (agregar productos, eliminar o vaciar el carrito) se gestionaron mediante peticiones POST seguidas de una redirección HTTP (Redirect 302) hacia la ruta GET principal (Index). Esto previene la molesta alerta de “Confirmar reenvío de formulario” en el navegador si el usuario presiona la tecla F5.

4. Reflexión y Decisiones Técnicas

A través del desarrollo de la práctica, se concluye que las sesiones del servidor son la herramienta correcta y natural para gestionar el estado en **aplicaciones web tradicio-**

nales multi-página (como las construidas en esta práctica con Razor, Thymeleaf o PHP crudo).

Sin embargo, para arquitecturas más modernas como una **API REST consumida por una SPA** (por ejemplo, el Proyecto de Fin de Ciclo en Angular), el mecanismo tradicional de sesiones pierde efectividad por problemas de escalado horizontal y acoplamiento. En esos escenarios, la decisión técnica correcta recae en el uso de **JWT (JSON Web Tokens) almacenado en una cookie HttpOnly**, lo que preserva la naturaleza *stateless* (sin estado) intrínseca de los servicios REST.